Intelligent OCR System Using Open-Source LLM

Technical Report - AI/ML Engineer (Agentic AI) Assessment

Submitted to: Micro Integrated Semiconductor Systems Pvt. Ltd. (Job Code 2338)

Prepared by: Akshay Chavan

1. Introduction

This report explains the OCR + LLM document understanding system I built for this assignment. The goal given was to make a tool that can take a document image or PDF, read the text out of it, understand what kind of document it is, pull out the important fields, check whether those fields look valid, and answer simple questions about the document - all using open-source models only, no paid APIs.

I built the whole pipeline in Python with a Streamlit front end. I tried to keep the project modular so each part (preprocessing, OCR, LLM, classification, validation) sits in its own file, which made debugging a lot easier once things started breaking, which they did, more than once.

2. OCR Workflow

2.1 Preprocessing

Before running OCR, the image goes through a preprocessing step using OpenCV. I do grayscale conversion first, then denoising, then adaptive thresholding to get a cleaner black-and-white version of the text. I also added a deskew step using minAreaRect on the thresholded pixels to straighten out images that were scanned or photographed at a slight angle, since a lot of the sample Aadhaar/PAN images I tested with were not perfectly straight. Finally I apply a basic sharpening kernel which seemed to help EasyOCR pick up smaller text like address lines.

Initially I skipped the deskew step thinking it wasn't needed for clean scans, but once I tested with a photo taken on my phone (slightly tilted) the OCR accuracy dropped a lot, so I added it back in.

2.2 OCR Extraction

For the actual text extraction I used EasyOCR as the primary engine, running on GPU when available. EasyOCR returns text along with bounding boxes and a confidence score per detected line, which I use later for the bounding box visualization and for an overall average confidence metric shown in the UI.

I added Tesseract as a fallback for cases where EasyOCR fails or returns a very low confidence (below about 15%). This happened a couple of times in my testing with low-resolution images, and having a second engine to fall back on made the pipeline more reliable than depending on just one OCR library.

2.3 PDF Support

Since the task also mentions PDF input, I added a small module using PyMuPDF (fitz) that converts each page of an uploaded PDF into a PNG at 300 DPI before sending it through the same preprocessing and OCR pipeline. Right now only the first page gets processed automatically in the UI; multi-page handling is something I'd extend given more time.

3. LLM Integration

For the language model part I used Qwen2.5-3B-Instruct, pulled from Hugging Face. I picked this model mainly because it's small enough to run reasonably on a single GPU (or even CPU for testing, though slowly) while still being instruction-tuned well enough to follow a structured prompt.

The LLM is doing two jobs in one prompt call: cleaning up the raw OCR text (fixing obvious spacing issues and broken words from misreads) and pulling out structured fields as JSON, based on whatever document type was detected. I originally had these as two separate LLM calls but merged them into a single prompt later on since two calls roughly doubled the response time for not much extra benefit.

The prompt asks the model to return its answer in a fixed format (a CORRECTED: section followed by a JSON: section) so I can split the response with simple string parsing rather than relying on the model to always produce perfectly clean JSON on its own. In practice the model mostly cooperates, but I still wrap the JSON parsing in a try/except and fall back to returning the raw text if parsing fails, because occasionally it adds a stray sentence before the JSON.

4. Information Extraction Approach

The fields extracted depend on the document type, since an Aadhaar card and an invoice don't share much in common. I give the LLM a general list of possible field names (name, date of birth, document number, address, phone, email, invoice number, total amount, date, issuer) and let it decide which ones actually apply, filling in null for anything not present rather than guessing values that aren't there.

Document classification itself is done before the LLM step, using a simple keyword-scoring approach over the OCR text rather than a trained classifier. Each document category (Aadhaar, PAN, Resume, Invoice, etc.) has a small list of keywords, and whichever category matches the highest proportion of its keywords gets selected, with a rough confidence percentage based on how many keywords matched. It's not as robust as a trained model would be, but for the scope of this assignment and given the open-source/no-paid-API constraint, it gave reasonably sensible results on the sample documents I tried it on.

5. Validation Logic

Once fields are extracted, each one is checked against a regex-based validator if a rule exists for that field name. I implemented validators for:

- Email - standard email pattern
- Phone - 10-digit Indian mobile format starting with 6-9
- PAN - 5 letters, 4 digits, 1 letter
- Aadhaar - 12 digit numeric format only (this is just a format check, not a real verification against UIDAI)
- GST number - standard 15-character GSTIN pattern
- IFSC code - 4 letters, 0, then 6 alphanumeric
- Date fields - a few common date formats (DD/MM/YYYY, DD-MM-YYYY, YYYY-MM-DD)

- Numeric fields like total amount - just checks it can be parsed as a number

Fields without a matching validator are shown as 'no validator defined' rather than marked invalid, since I didn't want the UI to wrongly flag something as wrong just because I hadn't written a rule for it yet.

6. Document Chat

The last part of the system lets the user type a question about the uploaded document, and the same LLM answers it using only the corrected OCR text as context (not the original image). This is a fairly direct retrieval-style prompt rather than anything fancy like a vector store, which felt like overkill for a single-document use case but would probably be the next thing to add if this needed to handle a folder of documents instead of one at a time.

7. Challenges Faced

A few things took longer than expected to get working:

- EasyOCR occasionally returned result tuples with fewer than 3 elements, which crashed the text-joining step with an index error until I added a length check before unpacking.
- Running the LLM on CPU for local testing was extremely slow (the 3B model download alone took over 30 minutes on a normal home connection), so most of my real testing ended up happening on a Kaggle GPU notebook instead of my laptop.
- Getting the model to consistently return parseable JSON took a few rounds of prompt tweaking. Lowering the temperature and giving an explicit output format helped more than I expected.
- The keyword-based classifier sometimes confuses Bank Passbook and Bank Statement since they share a lot of vocabulary (IFSC, account, branch). I'm aware of this and would address it with a proper trained classifier if I had more time, or at minimum a tie-breaking rule based on which keywords are more distinctive per category.
- Deskewing occasionally over-rotated already-straight images by a degree or two, so I added a minimum angle threshold (0.5 degrees) before applying rotation, instead of always rotating by whatever angle was detected.

8. Conclusion

Overall the system covers all the core requirements - preprocessing, OCR with a fallback engine, LLM-based correction and structured extraction, classification with a confidence score, format-based validation, and a basic document chat feature - using only open-source components throughout (EasyOCR, Tesseract, OpenCV, PyMuPDF, and Qwen2.5-3B-Instruct via Hugging Face Transformers). Given more time I'd want to improve the classifier with an actual trained model instead of keyword matching, add multi-page PDF handling in the UI, and test against a wider variety of real document samples to tighten up the field extraction prompts further.